



CURSO DE TECNOLOGIA EM SISTEMAS PARA INTERNET

Estudante: Vinícius Amorim Santos
Matrícula: 232016600071

Estudante: Isabelle Cristine Nunes Silva
Matrícula: 242016600018

Data: 12/06/2026 Período/Semestre: 2026.1

Componente Curricular: INTRODUÇÃO À
INTELIGÊNCIA ARTIFICIAL

Profº: Fábio Henrique Monteiro Oliveira

Implementação de Funções Lógicas Utilizando Retropropagação do Erro

1. Introdução

As Redes Neurais Artificiais (RNAs) constituem uma das principais abordagens da área de Inteligência Artificial, sendo amplamente utilizadas para reconhecimento de padrões, classificação e aproximação de funções. Entre os diversos modelos existentes, destaca-se o Perceptron Multicamadas (MLP), capaz de resolver problemas linearmente e não linearmente separáveis por meio do treinamento supervisionado utilizando o algoritmo de retropropagação do erro (Backpropagation).

Um dos problemas clássicos empregados para avaliar a capacidade de aprendizado das redes MLP é a implementação da função lógica XOR (OU Exclusivo). Diferentemente das funções AND e OR, o XOR não é linearmente separável, exigindo a utilização de pelo menos uma camada intermediária para que a rede consiga representar adequadamente sua fronteira de decisão.

Este trabalho apresenta a implementação de uma rede neural MLP desenvolvida em Python para aprendizagem de funções lógicas utilizando o algoritmo Backpropagation. Além da

reprodução do experimento XOR, foram realizados testes variando a quantidade de neurônios da camada oculta, a taxa de aprendizado e a inicialização dos pesos, permitindo analisar a influência desses parâmetros no desempenho da rede.

2. Fundamentação Teórica

As Redes Neurais Artificiais são compostas por neurônios interconectados organizados em camadas de entrada, ocultas e saída. O conhecimento adquirido durante o treinamento é armazenado nos pesos sinápticos que conectam os neurônios.

O treinamento de uma rede MLP ocorre por meio do algoritmo Backpropagation, que realiza inicialmente a propagação direta dos dados (forward propagation), calculando as saídas da rede. Em seguida, o erro obtido é propagado para trás (backward propagation), permitindo o ajuste dos pesos através do método do gradiente descendente.

Para introduzir não linearidade ao modelo, foram consideradas as funções de ativação Sigmóide Binária, Sigmóide Bipolar e Tangente Hiperbólica. Entretanto, a implementação da rede utilizou a função sigmóide binária tanto na camada oculta quanto na camada de saída.

O desempenho da rede foi avaliado por meio do Erro Quadrático Médio (Mean Squared Error - MSE), utilizado como critério de parada durante o treinamento.

3. Metodologia

O desenvolvimento foi realizado na linguagem Python utilizando as bibliotecas NumPy para operações matriciais e Matplotlib para geração de gráficos.

O estudo foi dividido em quatro etapas:

1. Implementação das funções de ativação e suas derivadas;
2. Desenvolvimento da arquitetura MLP e do algoritmo Backpropagation;
3. Reprodução do experimento XOR;
4. Realização de experimentos complementares variando parâmetros da rede.

Os testes foram conduzidos utilizando os padrões clássicos da função XOR, compostos por

quatro combinações binárias de entrada e suas respectivas saídas esperadas.

Durante o desenvolvimento do projeto, ferramentas de Inteligência Artificial Generativa foram utilizadas como apoio para aprimoramento da redação do relatório, organização da estrutura do código e esclarecimento de dúvidas relacionadas à implementação. As sugestões fornecidas foram analisadas e adaptadas pelos autores, permanecendo sob sua responsabilidade todas as decisões de desenvolvimento, validação e interpretação dos resultados.

4. Implementação

A rede neural foi implementada por meio da classe MLP, composta por dois neurônios de entrada, uma camada oculta com quantidade variável de neurônios e um neurônio de saída. Os pesos e vieses foram inicializados aleatoriamente em valores compreendidos entre -1 e 1.

A fase forward calcula inicialmente a ativação da camada oculta através do produto matricial entre entradas e pesos, seguido da aplicação da função sigmóide. Em seguida, a saída da rede é obtida utilizando procedimento semelhante na camada de saída.

Na fase backward é calculado o erro da camada de saída e realizada sua propagação para a camada oculta. Os pesos e vieses são então atualizados utilizando a taxa de aprendizado definida para o treinamento. O processo é repetido até que o erro médio quadrático atinja a tolerância estabelecida ou seja alcançado o limite máximo de épocas.

Além da reprodução do XOR, foram implementadas rotinas automáticas para experimentos envolvendo quantidade de neurônios ocultos, taxa de aprendizado, inicialização dos pesos e aprendizagem das funções AND, OR, NAND e NOR.

5. Resultados

No experimento principal, a rede neural foi configurada com 4 neurônios na camada oculta, taxa de aprendizado de 0,2 e tolerância de erro de 0,001. O treinamento convergiu após 8.360 épocas, atingindo um erro final de 0,001.

A análise do gráfico de treinamento mostra uma redução contínua do Erro Quadrático Médio

(MSE) ao longo das épocas. Inicialmente, o erro apresentava valores próximos de 0,31, diminuindo gradualmente até atingir o critério de parada estabelecido. Observa-se uma queda mais acentuada do erro entre aproximadamente 1.000 e 2.000 épocas, seguida por uma redução mais lenta e estável até a convergência.

As saídas produzidas pela rede para a função XOR foram 0,0280, 0,9663, 0,9690 e 0,0334 para as entradas (0,0), (0,1), (1,0) e (1,1), respectivamente. Esses valores ficaram muito próximos das saídas esperadas, demonstrando que a rede conseguiu aprender corretamente o comportamento da função lógica XOR.

6. Discussão

Os resultados obtidos demonstram a capacidade da rede MLP em resolver problemas não linearmente separáveis. A função XOR, tradicionalmente utilizada como benchmark em redes neurais, foi aprendida com sucesso pela arquitetura implementada.

Observou-se que a quantidade de neurônios na camada oculta influencia diretamente o desempenho do treinamento. Entretanto, o aumento do número de neurônios não resultou necessariamente em convergência mais rápida, evidenciando que arquiteturas maiores nem sempre produzem melhores resultados.

A taxa de aprendizado apresentou forte impacto na velocidade de convergência. Valores mais elevados permitiram alcançar o erro desejado em menor número de épocas, enquanto valores menores ocasionaram treinamentos mais lentos.

A variação dos pesos iniciais também afetou o processo de treinamento, indicando que diferentes pontos de partida podem conduzir a trajetórias distintas de otimização.

7. Conclusões

A implementação desenvolvida permitiu reproduzir com sucesso o treinamento de uma rede neural MLP utilizando o algoritmo Backpropagation para aprendizagem de funções lógicas.

A rede apresentou desempenho satisfatório na resolução da função XOR, confirmando a importância das camadas ocultas para problemas não linearmente separáveis. Os

experimentos realizados também evidenciaram a influência da taxa de aprendizado, da arquitetura da rede e da inicialização dos pesos no processo de convergência.

Dessa forma, o trabalho contribuiu para a compreensão prática dos mecanismos de treinamento de redes neurais artificiais e da aplicação do algoritmo de retropropagação do erro.

8. Referências

BRAGA, A. P.; CARVALHO, A. C. P. L. F.; LUDERMIR, T. B. Redes Neurais Artificiais: Teoria e Aplicações. Rio de Janeiro: LTC, 2000.

FAUSETT, L. Fundamentals of Neural Networks: Architectures, Algorithms and Applications. New Jersey: Prentice Hall, 1994.

OLIVEIRA, F. H. M.; BRAGA, P. H. C. Implementação de Funções Lógicas Utilizando Retropropagação do Erro. Universidade Federal de Uberlândia, 2011.

GOOGLE. Gemini. Modelo de inteligência artificial generativa. Disponível em: <https://gemini.google.com/>. Acesso em: 12 jun. 2026.